DevOps Laboratory Work

Linux Git Docker and Jenkins

Student: Kazbek Assanbek

Group: IT2-2302

Student ID: 37765

Course: DevOps

Report date: 13 October 2026

This report documents the real implementation and verified evidence for an individual DevOps laboratory project. It distinguishes command/API records from missing visual screenshots.

1. Student Information

Field	Value
Name	Kazbek
Surname	Assanbek
Group	IT2-2302
Student ID	37765
Course	DevOps

Personalized names: project directory assanbek_kazbek; repository RealKazbek/assanbek-kazbek-devops; image assanbek-kazbek-devops; container assanbek-kazbek-container; Jenkins Job Assanbek_Kazbek_DevOps.

2. Introduction and Objective

This individual laboratory implements one connected workflow: Linux, Git, GitHub, Docker, Jenkins, testing and Docker deployment. Linux-only commands were executed in a genuine Linux Docker environment. The GitHub repository is public and the Jenkins console records a real successful run.

This report does not invent screenshots. It cites genuine command and API evidence and explicitly identifies visual evidence that still must be captured manually.

3. Linux Tasks 1 to 5

Task 1 - Project Environment

Instructor requirement: Create the personalized project structure, student information file, configuration and log files. Demonstrate pwd, ls, cd, mkdir, touch, cp, mv, rm, cat, head and tail.

Implementation and actual result: The project contains app, scripts, config, logs and backup. Assanbek_Kazbek_info.txt contains the required five student fields. The Linux session demonstrates every required command and docs/OPERATIONS.md explains each one.

Evidence: evidence/linux/linux_commands.txt

Task 2 - Search and File Processing

Instructor requirement: Create application.log with at least ten messages; use find, grep ERROR, grep WARNING, an ERROR count and tail -n 5.

Implementation and actual result: logs/application.log contains realistic messages. The recorded Linux session finds the file, finds ERROR and WARNING lines, counts two ERROR lines and prints the last five entries.

Evidence: logs/application.log; evidence/linux/linux_commands.txt

Task 3 - File Permissions

Instructor requirement: Create private.txt and public.txt with different permissions. Use chmod and ls -l, then explain r, w, x, owner, group and others.

Implementation and actual result: private.txt was set to 600 and public.txt to 644. The Linux output shows -rw------- and -rw-r--r--; the operational notes explain why the files differ.

Evidence: private.txt; public.txt; docs/OPERATIONS.md; evidence/linux/linux_commands.txt

Task 4 - Bash Script

Instructor requirement: Create an executable personalized system script. It must show student and system data using whoami, hostname, date, uname, df and free, and check the information file.

Implementation and actual result: scripts/Assanbek_Kazbek_system.sh is executable. Its genuine Linux run prints all requested fields, disk and memory data, and confirms that the information file exists.

Evidence: scripts/Assanbek_Kazbek_system.sh; evidence/linux/linux_commands.txt

Task 5 - Processes

Instructor requirement: Demonstrate ps, ps aux and top. Identify PID, owner, CPU and memory, and explain process management and the difference between ps and top.

Implementation and actual result: The Linux record includes ps, ps aux and top output. ps aux shows PID, USER, %CPU and %MEM. docs/OPERATIONS.md gives the required explanation.

Evidence: docs/OPERATIONS.md; evidence/linux/linux_commands.txt

4. Git and GitHub Tasks 6 to 10

Task 6 - Git Repository

Instructor requirement: Initialize Git, check user.name and user.email, and show git status.

Implementation and actual result: The repository has a real initial commit and current Git identity is Kazbek / 37765@iitu.edu.kz. The repository was clean before this report update.

Evidence: local .git metadata; evidence/github/git_history_latest.txt

Task 7 - Meaningful Commit History

Instructor requirement: Create at least five meaningful commits and show git log and git log --oneline.

Implementation and actual result: The graph includes project initialization, application, Linux verification, Docker verification, Compose verification, documentation, Jenkins configuration and final verification commits. These are substantive changes rather than empty commits.

Evidence: evidence/github/git_history_latest.txt

Task 8 - Gitignore

Instructor requirement: Create .gitignore for *.log, backup/ and .env. Show ignored files and explain why the file is useful.

Implementation and actual result: The required three patterns are present. git check-ignore -v recorded matches for logs/application.log, backup and .env. The explanation notes that logs, backups and local secrets should not enter history.

Evidence: .gitignore; evidence/gitignore_check.txt

Task 9 - Branching and Merge

Instructor requirement: Create development, make a meaningful change, commit it, merge it into main, show the branch/graph and explain branches.

Implementation and actual result: development contains documentation commit e602227 and was merged by e5b03c7. The graph is preserved in the latest Git evidence.

Evidence: docs/OPERATIONS.md; evidence/github/git_history_latest.txt

Task 10 - GitHub

Instructor requirement: Create a repository, connect origin, verify remote, push main and ensure the complete project is on GitHub.

Implementation and actual result: GitHub CLI verified the public repository RealKazbek/assanbek-kazbek-devops and main branch. Local HEAD and origin/main matched at the time of verification.

Evidence: evidence/github/repository_verification.json; evidence/github/git_history_latest.txt

5. Docker Tasks 11 to 15

Task 11 - Dockerfile

Instructor requirement: Create a Dockerfile with FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. Run an application that displays the student information and success message.

Implementation and actual result: The Python standard-library application produces the requested text. Dockerfile uses python:3.13-alpine, /app, a non-root student user, port 8080 and python -m app.app.

Evidence: Dockerfile; app/app.py; tests/test_app.py; evidence/docker/docker_verification.txt

Task 12 - Build and Run Docker Container

Instructor requirement: Build the personalized image, run the named container, and demonstrate docker images, docker ps and docker ps -a.

Implementation and actual result: The Docker record proves a successful build of assanbek-kazbek-devops and a running assanbek-kazbek-container. Current Docker inspection also found the container running.

Evidence: evidence/docker/docker_verification.txt

Task 13 - Environment Variables

Instructor requirement: Pass STUDENT_NAME, STUDENT_SURNAME, STUDENT_GROUP and STUDENT_ID to the container. Show changed output and explain why variables are useful and secrets must not be hard-coded.

Implementation and actual result: app/app.py reads all four values. A real override run printed Changed Student, DEMO-1 and 99999. The operational notes explain runtime configuration and secret handling.

Evidence: app/app.py; tests/test_app.py; docs/OPERATIONS.md; evidence/docker/docker_verification.txt

Task 14 - Container Management

Instructor requirement: Demonstrate docker logs, stop, start, rm and, if applicable, exec. Explain image versus container.

Implementation and actual result: The evidence shows logs, exec, stop and start for the required container and an isolated removal demo. The notes correctly distinguish an immutable image from a running or stopped container instance.

Evidence: evidence/docker/docker_verification.txt; evidence/docker/compose_and_management.txt; docs/OPERATIONS.md

Task 15 - Docker Compose

Instructor requirement: Create docker-compose.yml, start the application with docker compose up, check it, stop it with docker compose down and explain Compose.

Implementation and actual result: The compose service supplies all student environment values, uses a non-conflicting container name, and was built, started, logged and removed through compose down.

Evidence: docker-compose.yml; evidence/docker/compose_and_management.txt; docs/OPERATIONS.md

6. Jenkins Tasks 16 to 20

Task 16 - Jenkins Job

Instructor requirement: Create the exact Jenkins Job Assanbek_Kazbek_DevOps using the GitHub repository.

Implementation and actual result: A live authenticated Jenkins controller on 127.0.0.1:8082 contains Assanbek_Kazbek_DevOps. Its live SCM configuration uses the public URL, */main and Jenkinsfile, with no empty credentialsId element.

Evidence: jenkins/job-config.xml; evidence/jenkins/job_api.json; evidence/jenkins/console_build_3_current_main.txt

Task 17 - Jenkinsfile

Instructor requirement: Create a declarative Jenkinsfile with meaningful Checkout, Build, Test, Docker Build and Docker Run stages.

Implementation and actual result: Jenkinsfile defines all five stages. The live build compiled Python, ran three tests including HTTP endpoints, built the image, ran the container and verified its health endpoint.

Evidence: Jenkinsfile; evidence/jenkins/console_build_3_current_main.txt

Task 18 - Jenkins and GitHub

Instructor requirement: Configure Jenkins to check out the GitHub project, build it, test it, build a Docker image and run a Docker container.

Implementation and actual result: The current live build fetched the public repository, checked out b56d47ede92be4388865efa298ee8a41d23287ca from origin/main, then executed all required stages.

Evidence: evidence/jenkins/console_build_3_current_main.txt; evidence/jenkins/build_3_api.json

Task 19 - Jenkins Environment Variables

Instructor requirement: Configure all four student variables and display Student, Group and Student ID in Jenkins Console Output.

Implementation and actual result: The Jenkins environment block contains all four values. The current console prints Student: Kazbek Assanbek, Group: IT2-2302 and Student ID: 37765.

Evidence: Jenkinsfile; evidence/jenkins/console_build_3_current_main.txt

Task 20 - Successful Pipeline

Instructor requirement: Run the pipeline successfully and demonstrate Dashboard, job name, pipeline stages, console output, Docker image/container and the Git commit used by Jenkins.

Implementation and actual result: Live Jenkins build #2 executed Checkout, Build, Test, Docker Build and Docker Run and ended Finished: SUCCESS on b56d47e. Its API confirms result SUCCESS and origin/main SHA. Browser screenshots remain BLOCKED because an available browser-control surface was not provided; no images were fabricated.

Evidence: evidence/jenkins/console_build_3_current_main.txt; evidence/jenkins/build_3_api.json; evidence/jenkins/job_api.json; evidence/jenkins/controller_and_docker_verification.txt; evidence/SCREENSHOT_CAPTURE_STATUS.md

7. Integrated DevOps Workflow Task 21

Task 21 - Complete DevOps Workflow

Instructor requirement: Demonstrate Linux to Git to GitHub to Jenkins to build, test, Docker build, Docker run and SUCCESS. Explain each connection and failure handling.

Implementation and actual result: docs/OPERATIONS.md explains the actual implemented chain, image/container distinction and failure behavior. Linux, GitHub, Docker and Jenkins logs provide genuine technical evidence. The student still must present the workflow in the required own-voice video.

Evidence: docs/OPERATIONS.md; README.md; VIDEO_GUIDE.md; evidence/linux/; evidence/github/; evidence/jenkins/

8. Real Evidence Extracts

The extracts below come from real saved terminal and Jenkins console output. Full files remain in the project evidence directory.

Linux command and permissions evidence

```
Linux environment: Linux ... aarch64 Linux
find . -name application.log
./logs/application.log
-rw------- ... private.txt
-rw-r--r-- ... public.txt
Student information file exists.
ps aux ... USER PID %CPU %MEM
```

Docker and Compose evidence

```
assanbek-kazbek-devops:latest
assanbek-kazbek-container ... Up ... 8081->8080/tcp
Name: Kazbek
Surname: Assanbek
Group: IT2-2302
Student ID: 37765
Application is running successfully!
```

GitHub and branch evidence

```
* b56d47e (HEAD -> main, origin/main) Add HTTP health verification and test coverage
*   e5b03c7 Merge development documentation updates
| * e602227 (development) Document Linux and Docker operating procedures
RealKazbek/assanbek-kazbek-devops | PUBLIC | default=main
```

Jenkins successful pipeline evidence

```
Checking out Revision b56d47ede92be4388865efa298ee8a41d23287ca
[Pipeline] { (Checkout)
[Pipeline] { (Build)
[Pipeline] { (Test)
[Pipeline] { (Docker Build)
[Pipeline] { (Docker Run)
Student: Kazbek Assanbek
Group: IT2-2302
Student ID: 37765
Finished: SUCCESS
```

9. Screenshot Evidence Status

The assignment requires visual demonstration of GitHub and Jenkins. Browser-control was unavailable while this report was prepared, so no genuine screenshots were captured and no mockups or generated images are included.

The blocked list and manual capture instructions are stored in evidence/SCREENSHOT_CAPTURE_STATUS.md. Current textual/API evidence is evidence/jenkins/console_build_3_current_main.txt and evidence/jenkins/build_3_api.json. It records all five stages, current-main commit b56d47e and Finished: SUCCESS.

10. Conclusion

The Linux, Git, GitHub, Docker, Compose and Jenkins technical work is implemented and supported by real command evidence. The remaining submission work is the student's own continuous voice video and the capture of genuine graphical screenshots while presenting the existing project.

11. Final Verification Summary

Area	Result	Evidence
Linux Tasks 1 to 5	Verified by genuine Linux command output	evidence/linux/linux_commands.txt
Git and GitHub Tasks 6 to 10	Verified by repository graph, origin and GitHub CLI	evidence/github/
Docker Tasks 11 to 15	Verified by build, container and Compose records	evidence/docker/
Jenkins Tasks 16 to 19	Verified by live job configuration and current-main build	jenkins/job-config.xml; evidence/jenkins/
Task 20 visual evidence	Partial: textual/API success verified; screenshots are blocked	evidence/SCREENSHOT_CAPTURE_STATUS.md

12. Manual Submission Actions

The student must record one continuous 10 to 20 minute screen video with their own voice. During the recording, show the GitHub main branch and merge, Linux commands, Docker evidence, Compose, the Jenkins Job, pipeline stages, console output, Docker image/container and Finished: SUCCESS. Save only genuine screenshots taken from the displayed real interfaces.